

東京都オープンデータAPI

自治体データを活用する

rskmt

1

東京都の人口、150年前は？

- 2023年の東京都の人口は **14,099,993人**（約1410万人）

❗ 問題：明治5年（1872年）、東京には何人いた？

今の半分？ 3分の1？ 10分の1？

- 答え：**859,345人**
→ 今の約 **16分の1**
- しかも1872年から **152年分**、毎年の人口が誰でも取れる形で公開されている
→ **東京都オープンデータAPI**

2

オープンデータか

3

オープンデータとは

- 政府・自治体が **誰でも自由に使える形式** で公開するデータ
- 商用・非商用問わず **二次利用可能**
- 機械可読（CSV, JSON, API）で提供

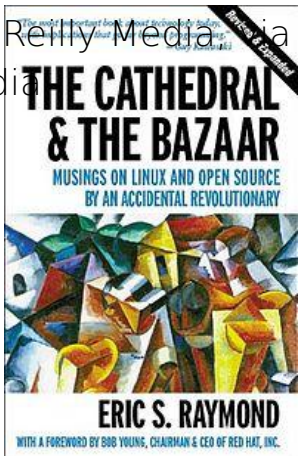
❗ 意義

- 透明性**：行政の活動を市民が検証できる
- イノベーション**：民間がデータを活用してサービス開発
- 効率化**：データに基づく政策立案（**EBPM: Evidence-Based Policy Making**）

4

「オープン」の源流

書影: O'Reilly Media
Wikipedia



全文を読む（山形浩生 訳） [🔗](#)

- 「オープン」の源流は **ソフトウェア**
→ **オープンソース**：コードを公開し、誰でも使える・直せるようにする
- 古典『**伽藍とバザール**』（*The Cathedral and the Bazaar*、1999）
→ 伽藍（＝大聖堂）方式：専門家だけで **閉じて** 作る
→ バザール方式：**全部公開して**、大勢で作る
→ 主張は **バザールの勝ち**（証拠は Linux）
- オープンデータは、この「オープン」を **行政データ** に広げたもの

5

配っても、減らない

- 「魔法のおなべ」(The Magic Cauldron、[全文](#)) は、オープンソースの 経済学

① 問い

コードを タダで配ったら、作った人はどうやって食べていく？

- カギは 非競合性 (non-rivalry) — パンは食べればなくなるが、コードは コピーしても減らない
 - 使う人・直す人が増えるほど改良が戻ってきて、鍋の中身はむしろ増える
- 売り物は別に作れる：サポート、ハードウェア、ブランド、コンテンツ
- データも非競合 — 配っても都のデータは減らず、使われた分だけ価値が生まれる

絵: John William Waterhouse, Public domain, via Wikimedia Commons



身近なオープンデータ活用例

サービス	使用データ
乗換案内アプリ	公共交通の時刻表・遅延情報
防災アプリ	避難所・ハザードマップ
不動産サイト	地価公示・用途地域
天気予報	気象庁の観測データ

- 実は意識せずに日常的に使用している
- 自分でもAPIを叩けば 同じデータ が手に入る
 - 登録が必要な場合もある

東京都オープンデータAPI

概要

- 東京都オープンデータカタログサイト の一部データをAPI提供
- ユーザ登録不要 — 誰でもすぐに利用可能
- 掲載データ約54,000件のうち、CSV約35,000件がAPI対応

項目	内容
認証	不要
レート制限	明示なし（常識的な範囲で）
データ形式	JSON / XML
CORS	対応済み

Wikipedia APIとの違い

	Wikipedia API	東京都API
HTTPメソッド	GET	POST
User-Agent	必須	不要
データ内容	百科事典・統計	行政・地理・施設
更新頻度	リアルタイム～日次	不定期（年次も多い）

⚠ 重要：今回は**GET**ではなく**POST**

- Wikipedia API は `requests.get()`
- 東京都API は `requests.post()`

10

東京都オープンデータAPIの構造

11

基本URL

```
1 https://service.api.metro.tokyo.lg.jp/api/{apiId}/{format}
```

- `{apiId}` — データセットごとに異なるID
- `{format}` — `json` または `xml`

例：東京都人口の推移（JSON）

```
1 https://service.api.metro.tokyo.lg.jp/api/t000003d2000000236-c054eb26d9e9721642bef78c7c717e76-0/json
```

12

APIのIDを探す

- API検索ページ** でキーワード検索
- 欲しいデータの `apiId` をコピーして使う

💡 探し方のコツ

- 「避難所」「駐輪場」「人口」などキーワードで検索
- 区市町村名で絞り込み可能
- 詳細ページで `apiId` と列名を確認

ただし、リンク切れや内容がないことも多いため確認が必要

13

基本的な使い方

14

まず、何もしないで呼ぶ

- パラメータも条件も付けず、**空のオブジェクト「{ }」** を **POST** する
→ **GET**ではない

```
Code
1 import requests
2
3 api_id = "t000003d2000000236-c054eb26d9e9721642bef78c7c717e76-0"
4 url = f"https://service.api.metro.tokyo.lg.jp/api/{api_id}/json"
5
6 response = requests.post(url, json={})
7 data = response.json()
8
9 print(data)
```

```
1 {'total': 152, 'subtotal': 100, 'limit': None, 'offset': None, 'metadata': {'apiId': 't000003d2000000236-c054eb26d9e9721642bef78c7c717e76-0'}}
```

💡 `json={}` は必須

- 空のJSONボディ送信が必須（東京都APIの仕様）
- 仮に省略すると **415**（Unsupported Media Type）が返ってくる

15

長いレスポンスデータの解析方法 I

手順1： まずkeysをみて全体の構造を確認

- レスポンスはマシン用なので人間はパッと見ではわかりにくい
→ そこで、一旦 レスポンスのdictのキーを表示してみる

Code

```
1 print(data.keys())
```

```
1 dict_keys(['total', 'subtotal', 'limit', 'offset', 'metadata', 'hits'])
```

手順2： キーの下を深掘り

- 大分見通しがよくなった
- 意外にもルートのキーは6個だけ
→ 各キーの下に情報が連なっている構造
- では、各キーの下をみていって何があるのかを理解しよう

Code

```
1
2 for key, datum in data.items():
3     print(key, type(datum), datum)
4     print("")
5     print("-----")
6     print("")
```

16

長いレスポンスデータの解析方法 II

手順3： 重要キーの同定

```
1 total <class 'int'> 152
2
3 -----
4
5 subtotal <class 'int'> 100
6
7 -----
8
9 limit <class 'NoneType'> None
10
11 -----
12
13 offset <class 'NoneType'> None
14
15 -----
16
17 metadata <class 'dict'> {'apiId': 't000003d2000000236-c054eb26d9e9721642bef78c7c717e76-0', 'title': '人口の動き（令和5...（略）
18
19 -----
20
21 hits <class 'list'> [{'row': 1, '和暦（年次）': '明治5', '西暦（年次）': 1872, '東京都／人口補足': ' ', '東京都／人口': 859345, ...（略）
22
23 -----
```

- キー **metadata** はデータセットの情報っぽい
→ メタデータは **データのデータ**
- キー **total** はデータの総件数っぽい
- どうも重要そうなのは **hits** っぽい
→ 正解：東京都APIの仕様では、 **hits** にデータ本体があることになっている

手順4： 重要キーの深掘り

- **hits** には大量のデータがlistで格納されていることがわかった
- よって、最初の1件目を出力して実データを確認することができそう

Code

```
1 print(data["hits"][0])
```

```
1 {'row': 1, '和暦（年次）': '明治5', '西暦（年次）': 1872, '東京都／人口補足': ' ', '東京都／人口': 859345, '東京都／増減': '...', '東京都／増減率': 0.0001}
```

- 1件目＝ **明治5年（1872年）**、人口 **859,345人**

17

hits を DataFrame にする

18

hits は DataFrame にそのまま入る

- hits は list of dict — pd.DataFrame() にそのまま渡せる（キーが列名になる）
- さっきの data に続けて：

```
Code
1 import pandas as pd
2
3 df = pd.DataFrame(data["hits"])
4 print(df.columns)

1 Index(['row', '和暦（年次）', '西暦（年次）', '東京都／人口補足', '東京都／人口', '東京都／増減', '東京都／増減率(%)',
2       '東京都／対全国比(%)', '全国／人口補足', '全国／人口(千人)', '全国／増減(千人)', '全国／増減率(%)'],
3       dtype='object')
```

- 知らないデータは、まず列名の確認から — Pandas入門
- 人口だけかと思ったら、増減率・対全国比・全国の人口まで 12列 あった
- row はAPIが付ける行番号
- 使う列は、この中から選ぶ

Pandasなので絞り込みも楽

- 列 の絞り込み（西暦（年次）など）
- DFやSeriesなどのサブセットにする
- 行 の絞り込み（期間指定など）
- 条件にあう行の抽出

```
Code
1 # 列：使う列だけ選ぶ
2 pop = df[["西暦（年次）", "東京都／人口"]]
3
4 # 行：受け取ってから、条件抽出で絞る
5 recent = df[df["西暦（年次）"] >= 1950]
```

💡（参考）jsonでPOSTすれば列の絞り込みがAPI上で可能

- 実は、bodyに column というキーでPOSTすれば絞られた状態で戻る — 必要な列だけ受け取れる

```
Code
1 body = {"column": ["西暦（年次）", "東京都／人口"]}
2 response = requests.post(url, json=body)
```

東京都人口（期間指定）

- 1960年代以降だけに絞って、高度経済成長期の東京を見る
- 列を選んでから、行を条件抽出 — さっきの2つの道具をそのまま使う

Code

```
1 import requests
2 import pandas as pd
3
4 api_id = "t000003d2000000236-c054eb26d9e9721642bef78c7c717e76-0"
5 url = f"https://service.api.metro.tokyo.lg.jp/api/{api_id}/json"
6
7 response = requests.post(url, json={})
8 data = response.json()
9
10 df = pd.DataFrame(data["hits"])
11 pop = df[["西暦（年次）", "東京都／人口"]]
12 sixties = pop[pop["西暦（年次）"] >= 1960]
13
14 print(sixties)
```

	西暦（年次）	東京都／人口
1		
2	88	1960 9683802
3	89	1961 9936970
4	90	1962 10180203
5	91	1963 10432526
6	92	1964 10639361
7	93	1965 10869244
8	94	1966 10973070
9	95	1967 11104969
10	96	1968 11251775
11	97	1969 11340417
12	98	1970 11408071
13	99	1971 11521226

1000万人を超えたのはいつ？

- 出力を眺めると、**1962年** に1000万人を超えている

統計分析の例

- 列を取り出せば **Series** — `.mean()` と同じ形で `.max()` `.std()` `.median()` も使える

→ いつもの統計がそのまま使える

Code

```
1 import requests
2 import pandas as pd
3
4 api_id = "t000003d2000000236-c054eb26d9e9721642bef78c7c717e76-0"
5 url = f"https://service.api.metro.tokyo.lg.jp/api/{api_id}/json"
6
7 response = requests.post(url, json={})
8 data = response.json()
9
10 df = pd.DataFrame(data["hits"])
11 population = df["東京都／人口"]
12
13 print(f"平均: {population.mean():,.0f}人")
14 print(f"最大: {population.max():,}人")
15 print(f"最小: {population.min():,}人")
16 print(f"標準偏差: {population.std():,.0f}")
17 print(f"中央値: {population.median():,.0f}人")
```

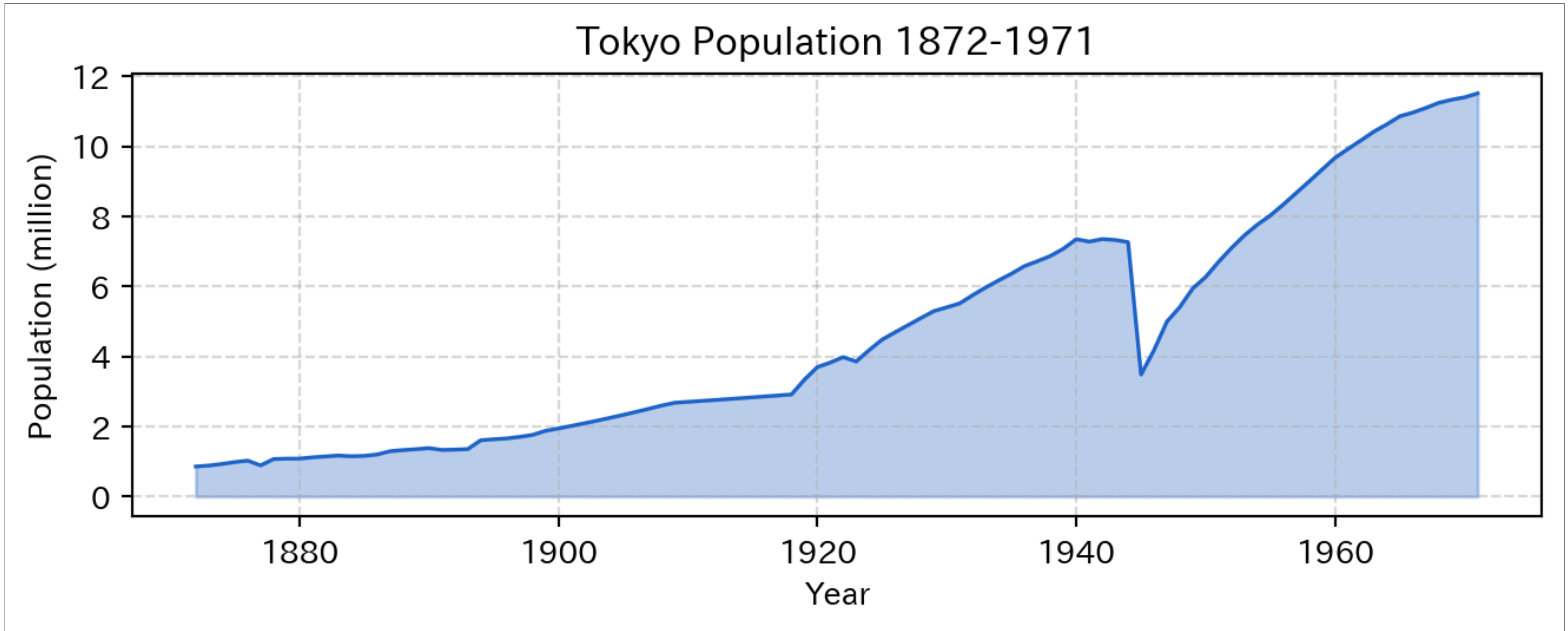
1	平均: 4,623,763人
2	最大: 11,521,226人
3	最小: 859,345人
4	標準偏差: 3,257,045
5	中央値: 3,765,064人

どんな形で増えた？

- 最小 **859,345人**（1872年）から最大 **11,521,226人**（1971年）まで、100個の値が並んでいる
- **データの可視化** の大原則 — *Always plot your data first.*

❗ ここで質問

100年分の折れ線は、どんな形？ まっすぐ右肩上がり？ 途中で事件はある？ — 手で描いてみてから、めくる



- 右肩上がり — ただし **一直線ではない**
 - 明治・大正のゆるやかな伸び、昭和以降の加速
- **深い谷が1つ**

▶ コードを見る

24

何があった

- 谷の底は **1945年**：7,271,001人（1944年）→ **3,488,284人**
 - 1年で **378万人減、ほぼ半減**

❗ ここで質問

1945年の東京で何が起きた？ いなくなった人は、いつ戻った？

- **空襲と疎開** — 戦争末期、東京から人が離れた
- 戻りも速い：1946年に **+69万人**、1947年に **+82万人**。5年後の1950年には600万人台まで回復
- 歴史の教科書の出来事が、**数列にそのまま刻まれている**

25

実データは汚れている

- 人口の列は綺麗だったが、統計分析のときの **df** で、同じ表の **増減率(%)** の列を見ると...

Code

```
1 print(df["東京都／増減率(%)"])
```

1 0 ...
2 1 3.26
3 2 5.09
4 3 5.75
5 4 4.2
6 ...
7 95 1.2
8 96 1.32
9 97 0.79
10 98 0.6
11 99 0.99
12 Name: 東京都／増減率(%), Length: 100, dtype: object

- 末尾の **dtype: object** に注目 — 数値の列なら **int64** / **float64** になる
→ 数値が **文字列**（**"3.26"**）で入っていて、データがない1872年は **"..."**
- 条件抽出で **"..."** の行を除き、**.astype(float)** で **列ごと数値に変換**（**float()** の列版）

Code

```
1 clean = df[df["東京都／増減率(%)"] != "..."]  
2 rates = clean["東京都／増減率(%)"].astype(float)  
3  
4 print(f"増減率の平均: {rates.mean():.2f}%")  # 増減率の平均: 2.99%
```

💡 dtype を確かめてから使う

- 外部APIのデータは **型が揃っているとは限らない**
→ 数値のつもりが文字列、欠損が **"..."** などの記号
- Series を **print** すると末尾に **dtype** が出る — 集計の前に見るクセをつける

知らないデータを覗く

別の窓口、同じテンプレート

- **api_id** を差し替えるだけで、届くデータが変わる（通信部分のコードはそのまま）
- 知らないデータでも手順は同じ — **DataFrameにして、まず列名**

Code

```
1 import requests  
2 import pandas as pd  
3  
4 api_id = "t131091d0000000001-3f9091b43cf1caa550e7152b5c2ba34c-0"  # 品川区の避難所  
5 url = f"https://service.api.metro.tokyo.lg.jp/api/{api_id}/json"  
6  
7 response = requests.post(url, json={})  
8 data = response.json()  
9  
10 df = pd.DataFrame(data["hits"])  
11  
12 print(f"品川区の避難所（総件数: {data['total']}件）")  
13 print(df.columns)  
14 print(df)
```

1 品川区の避難所（総件数: 25件）
2 Index(['row', '名称', '緯度', '経度'], dtype='object')
3 row 名称 緯度 経度
4 0 1 南大井シルバーセンター 35.591619 139.734238
5 1 2 春光福祉会 特別養護老人ホーム「ロイヤルサニー」 35.599962 139.722470
6 2 3 さくら会老人保健施設「ケアセンター南大井」 35.594534 139.733675
7 3 4 西大井シルバーセンター 35.600898 139.723070
8 4 5 八潮児童センター 35.599807 139.751633
9 ...



品川区のコード の **api_id** だけ差し替えて実行する。

1	避難所（東京都全域）	t000003d00000000093-0af9bfa497bc7be6ef83e5bfd1fdcfc6-0
2	都営バスの運賃	t000018d00000000044-3db21a77b0af1c54b411f79604444ca2-0

💡 予想してから試す

total はそれぞれ何件くらいだと思う？（品川区の避難所は25件だった） — 桁を予想してから実行

まとめ

やりたいこと	コード
基本アクセス	<code>requests.post(url, json={})</code>
列の絞り込み（API側）	<code>body = {"column": ["列名1", "列名2"]}</code>
DataFrame化	<code>df = pd.DataFrame(data["hits"])</code>
列名の確認	<code>print(df.columns)</code>
行の絞り込み	条件抽出 <code>df[df["列名"] >= 値]</code>
全件数確認	<code>data["total"]</code>

⚠ Wikipedia API との違い

- **GET** ではなく **POST**
- 空でも `json={}` が必要
- **User-Agent** は不要

演習

演習：避難所データの可視化



任意の区市町村の **避難所データ** を取得し、**件数** を表示せよ。

- （発展）緯度・経度があれば散布図で地図風にプロット

💡 ヒント

- **API検索ページ** で「避難所」を検索
- 複数の区のデータを比較しても面白い

▶ 解答例を見る（品川区の例 — apildは検索ページで差し替え）

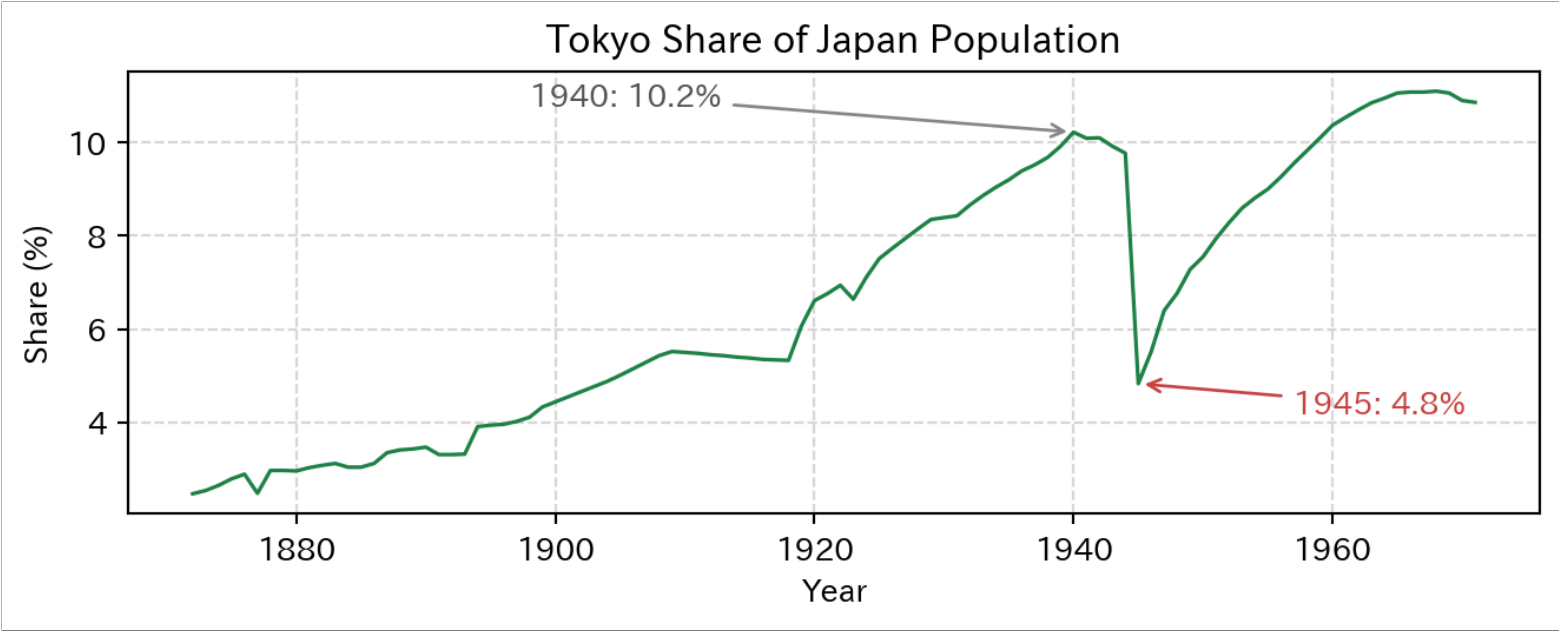
演習：東京一極集中はいつから？



人口データの「東京都／対全国比(%)」を折れ線グラフに描き、日本の人口の**1割**を最初に超えた年を答えよ。

- 1. 人口データを取得し、DataFrameにして列名を確認
- 2. 対全国比の値は文字列 — `.astype(float)` で数値化（増減率のときと同じ下ごしらえ）
- 3. 折れ線グラフに描き、10%を最初に超えた年を確認

▶ 解答例を見る



① 一極集中は戦前に始まっていた

- 1872年は **2.5%**（40人に1人）
- 最初に10%を超えたのは **1940年**
→ 戦前にすでに「1割が東京」だった
- 1945年に **4.8%** へ急落（人口の谷と同じ理由）し、1959年に再び10%へ

33

演習：2つのデータセットを比較



異なる2つのデータセット（例：2つの区の避難所、戦前と戦後の人口）を取得し、件数や統計量を比較して表示せよ。

- （発展）同じグラフに重ねて描画

▶ 解答例を見る（戦前と戦後の人口を比較する例）

34

活用アイデア

35

アプリケーション例

分野	活用例
防災	避難所マップ、消火栓・防火水槽の位置、地震被害想定
交通	駐輪場検索、バス運賃計算、駅の乗降客数分析
生活	公衆トイレマップ、公共施設案内、子育て支援制度
研究	人口動態分析、地価変動追跡、行政サービス利用調査

探してみる

API検索ページ で自分の住む区市町村を検索し、見つけたデータを1つ、隣の人に30秒で紹介する（何のデータ？ 何件？ どの列が面白そう？）

36

（参考）全件を取り寄せる

37

(参考) あれ？件数が足りない？

- 本編の人口データ、**hits** の件数と **total** をよく見ると：

Code

```
1 print(len(data["hits"]))
2 print(data["total"])
```

```
1 100
2 152
```

(参考) よく見ると **HITS** LIST長と **TOTAL** が違う

- 152年分あるのに、**100件**しか届いていない
→ 1回に届く件数は、指定しないと **先頭100件** まで

(参考) URLの後ろに **?LIMIT=200** を付ければ全件届く

Code

```
1 response = requests.post(url + "?limit=200", json={}) # limitはデフォルト100・最大1000
2 data = response.json()
3
4 print(len(data["hits"]))
```

```
1 152
```

(参考) 1,000件を超えたら — ページング

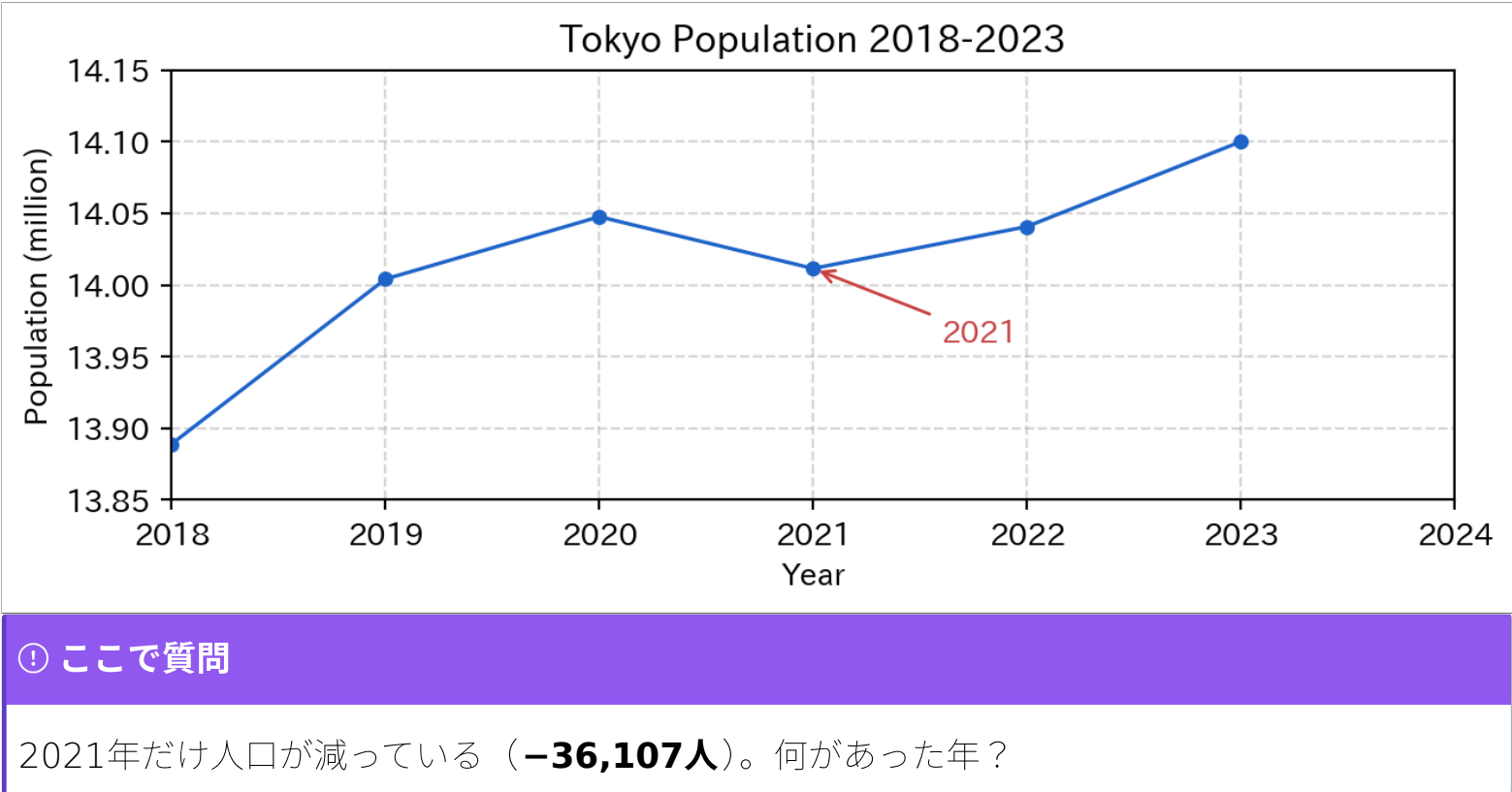
- 1,000件を超えるデータは、**limit** を最大にしても1回では取り切れない
- このとき、東京都APIでは取得開始位置 **offset** をずらしながら、同じリクエストを繰り返すことになっている
→ このようなデータの応答や表示の分割を **ページング** (paging) という
- たとえば **total: 2500** のデータなら3回に分ければとれる：

呼び出し	URLに付ける	届く範囲
1回目	?limit=1000&offset=0	1～1000件目
2回目	?limit=1000&offset=1000	1001～2000件目
3回目	?limit=1000&offset=2000	2001～2500件目

- 毎回の **hits** をつなげば全件そろう
- Webサービスの「次のページ」「もっと見る」も、裏側はこの仕組み

(参考) 全件取って、足元を拡大する

- 本編のグラフは1971年で切れていた — **?limit=200** で **152年全部** を取り直すと、2023年まで伸びる
- 「七夕」の8月29日を見つけたとき [☆]と同じ手 — **軸の範囲を絞って足元を拡大** する
→ グラフのコードに **plt.xlim(2018, 2024)** と **plt.ylim(13.85, 14.15)** を足すだけ



- **コロナ禍** — テレワークと転出で、人口が **1995年以来26年ぶり** に減った
- **ズームしないと見えない発見がある** (伝統的七夕のときと同じ)

関連リソース

リソース	URL
オープンデータカタログサイト	portal.data.metro.tokyo.lg.jp
APIカタログサイト	portal.data.metro.tokyo.lg.jp/opendata-api/
API検索ページ	spec.api.metro.tokyo.lg.jp/spec/search
公共交通オープンデータ	odpt.org
デジタル庁 GIF	digital.go.jp